

Conversation Storage and Memory Management

Snapshot: 2026-05-18

This note summarizes how user-agent conversations are recorded, replayed, cached, and bounded in the current AgenticSys_v2 backend after the memory-management optimization.

Summary

Conversation state is stored per case session in memory. The system does not maintain a single full permanent chat transcript file for replay. Instead, it splits conversation memory across four purpose-specific layers:

1. `CaseSession.input_history` for orchestrator replay.
2. `CaseSession.specialist_kb` for distilled cross-turn specialist memory.
3. `CaseSession.qa_cache` for recent duplicate-question answer replay.
4. `logs/<session_id>.jsonl` for operational audit/debug events.

The recent conversation remains available exactly for short-term continuity, while older heavy tool outputs are compacted or moved into structured knowledge points.

1. Orchestrator Replay Memory

Field: `CaseSession.input_history` in `server.py`.

This is the main conversation replay state passed back into the orchestrator on later turns. Each successful streamed run is captured from:

```
streamed.to_input_list()
```

That list can include:

- user messages
- orchestrator tool-call records
- specialist tool outputs
- assistant/final-answer messages

After each successful turn, `_prune_input_history(...)` runs.

Current behavior:

- The most recent 2 reviewer turns are kept intact.
- Older turns keep their message structure and function-call records.
- Older `function_call_output.output` payloads are replaced with a short elision stub.

This prevents large specialist outputs from being replayed on every later turn, while preserving enough structure for the orchestrator to know what happened.

2. Specialist Knowledge Base

Field: `CaseSession.specialist_kb` in `server.py`.

This is the main semantic memory layer. After a specialist completes, a distiller agent extracts structured `KnowledgePoint` entries from the specialist output. These are stored per specialist:

```
{
  "modeling": [KnowledgePoint, KnowledgePoint, ...],
  "spend_payments": [KnowledgePoint, KnowledgePoint, ...]
}
```

The KB is append-only during a session. Older entries are retained for audit, but the active memory shown to specialists is compact:

- latest knowledge point per topic wins
- each specialist receives only its own active digest
- the digest is prepended before a new specialist sub-question

This lets specialists reuse established findings without forcing the orchestrator to replay raw historical tool output.

3. QA Cache

Field: `CaseSession.qa_cache` in `server.py`.

This cache stores recent question-to-answer results for exact or near-duplicate questions. It is keyed by normalized redacted question text.

Current behavior after optimization:

- Cache hits skip the orchestrator run and replay the prior answer.
- Access refreshes insertion order, giving LRU behavior.
- The cache is bounded by `QA_CACHE_MAX_ENTRIES`.
- Default cap: 64 entries.

This cache is a speed optimization, not the audit source. Bounding it prevents long sessions from retaining every answer payload indefinitely.

4. Event Logs

Class: `EventLogger` in `logger/event_logger.py`.

Operational events are written to:

`logs/<session_id>.jsonl`

These logs include events such as:

- session open
- screening phases
- cache hits and cache stores

- orchestrator start/done
- tool calls and tool results
- input-history pruning stats
- KB warmth hints
- distiller successes/failures
- chart events
- errors and fallback behavior

The event log is useful for debugging and audit, but it is not a clean user-facing transcript.

5. Live SSE Delivery

Endpoint: `GET /api/cases/<case_id>/stream`.

Each connected frontend client receives live server-sent events through a per-client queue.

Current behavior after optimization:

- Each subscriber queue is bounded by `SSE_QUEUE_MAXSIZE`.
- Default cap: 256 events.
- If a client is slow or stale and its queue fills, the oldest pending event is dropped and the newest event is retained.

This prevents disconnected or slow clients from accumulating unbounded event backlogs in memory.

6. In-Turn Specialist History

Field: `AppContext._specialist_histories` in `agent_factories/app_context.py`.

Within a single outer orchestrator turn, each specialist can keep its own transcript so retries or follow-up tool calls do not start from scratch.

Current behavior after optimization:

- The latest 2 user-message windows are kept intact.
- Older in-turn specialist tool outputs are replaced by a compact elision stub.
- This applies only to the per-turn `AppContext`, not the cross-turn KB.

This reduces memory retained during retries while preserving enough context for local continuity.

7. Reset Behavior

Endpoint: `POST /api/cases/<case_id>/rewind`.

The rewind/reset path clears:

- `sess.input_history`
- `sess.qa_cache`
- `sess.specialist_kb`

It does not delete:

- JSONL event logs
- rendered chart files under `reports/<case_id>/charts/`

Net Effect

After the memory optimization:

- recent conversation context is preserved exactly for the last 2 turns
- older raw specialist payloads are replaced by stubs
- durable session knowledge is stored as compact specialist KB entries
- duplicate-answer replay is capped with LRU eviction
- live stream queues are capped
- in-turn specialist transcripts are compacted

This keeps functionality and answer quality intact while preventing the largest memory structures from growing without bounds.